

Conectividad y Redes entre entornos híbridos

Sesión 3 · Tus contenedores dejan de vivir solos

Curso 101-EAN · Gestión de Contenedores en Entornos de Nube Híbrida

Docente: Jawy Andrés Romero Pinto

MSc · Especialista en ciberseguridad y nube híbrida · LFCE

De la Sesión 2 a hoy

Tu imagen ya viaja por el mundo — pero viaja sola

Lo que ya sabes hacer

- Construir TU imagen con un Dockerfile (S2).
- Publicarla en Docker Hub y correrla donde sea (S2).
- Correr contenedores sueltos con puertos (S1).

Hoy (S3)

- **Las apps reales son VARIOS servicios que se hablan.**
- Creas redes propias donde los contenedores se encuentran por NOMBRE.
- Publicas al exterior SOLO lo necesario.
-  Y segmentas: la base de datos, invisible desde afuera.

Entregable de hoy: una aplicación multi-contenedor (blog + base de datos) corriendo en una red aislada.

Objetivos de la sesión

Al final de estas 2 horas podrás...

Explicar

cómo se comunican los contenedores: redes, DNS interno y mapeo de puertos.

Conectar

varios contenedores en una red propia donde se encuentran por su nombre.

Publicar

hacia el exterior únicamente los puertos que el servicio necesita exponer.

Aislar

los servicios sensibles: la base de datos accesible para la app, invisible para el mundo.

Una aplicación real nunca es UN contenedor

El patrón de toda app moderna



Separadas por diseño (S2: una imagen, una responsabilidad) — pero entonces... ¿cómo se encuentran?

¿Cómo sabe la web dónde está la base de datos?

Las IPs de los contenedores cambian en cada arranque.

¿Quién puede hablar con quién?

La BD no debería ser alcanzable desde Internet — nunca.

¿Qué se expone al exterior?

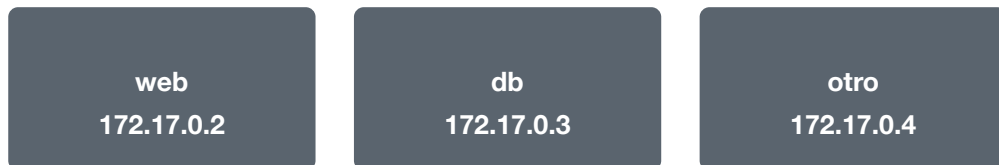
Solo la puerta de entrada; el resto es tráfico interno.

Las tres preguntas tienen la misma respuesta: redes de contenedores.

Lo que pasa cuando haces docker run

La red bridge por defecto — y por qué no basta

Red «bridge» (la de fábrica)



- Todos los contenedores caen aquí juntos, revueltos.
- Solo se alcanzan por IP — y la IP cambia en cada arranque.
- Sin resolución por nombre: «web» no sabe quién es «db».
- Cero segmentación: cualquier contenedor ve a cualquier otro.

El diagnóstico

La red de fábrica sirve para el contenedor suelto de la S1.

Para una aplicación de varios servicios necesitas:

1. Nombres estables (DNS interno)
2. Aislamiento entre grupos de contenedores
3. Control de qué se expone afuera

La solución: crear TUS redes. Una línea de comando — y las tres cosas llegan solas.

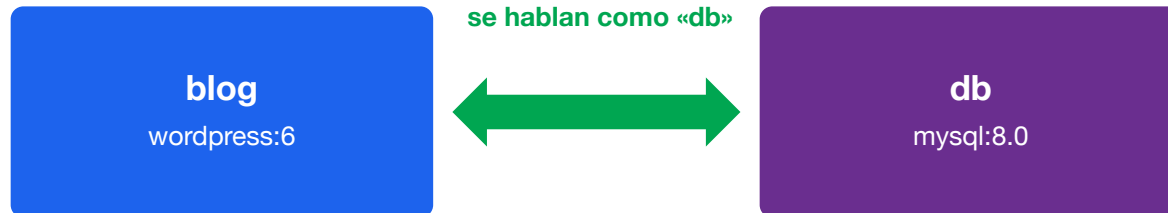
Redes definidas por ti: el superpoder del DNS

`docker network create` — y los contenedores se llaman por su nombre

```
docker network create ean-net
docker run -d --name db --network ean-net mysql:8.0
docker run -d --name blog --network ean-net -p 8080:80 wordpress:6
```



Red «ean-net» (tuya, aislada)

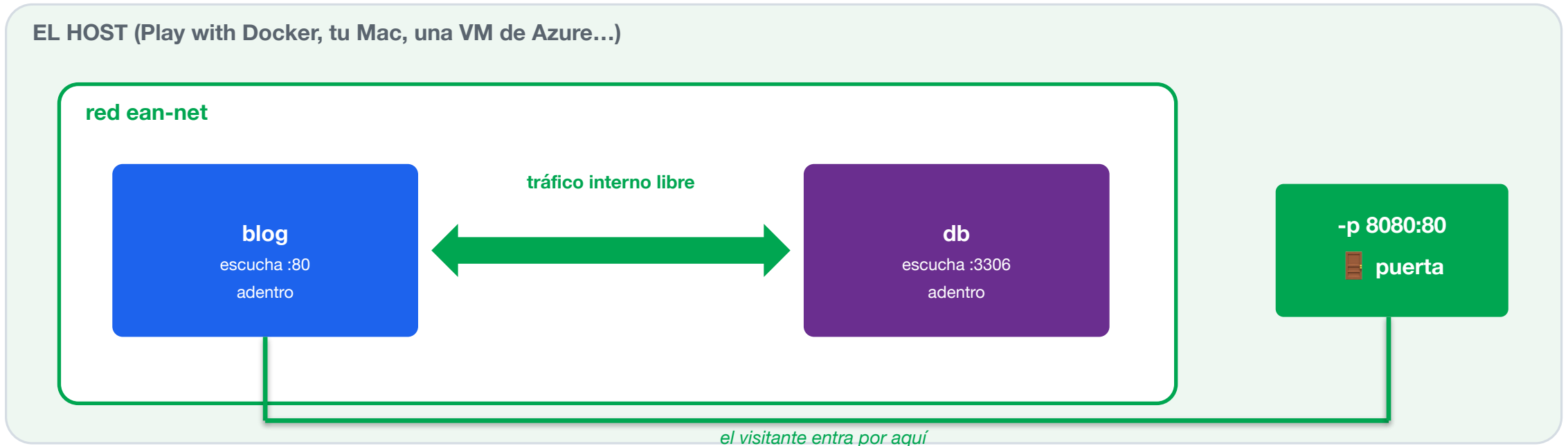


Qué acaba de pasar

- Docker montó un DNS interno en tu red.
- «db» resuelve SIEMPRE al contenedor db, cambie o no su IP.
- **La app se configura con nombres, no con IPs.**

Puertos: la puerta entre dos mundos

Dentro de la red todo fluye; hacia afuera, solo lo que TÚ publicas



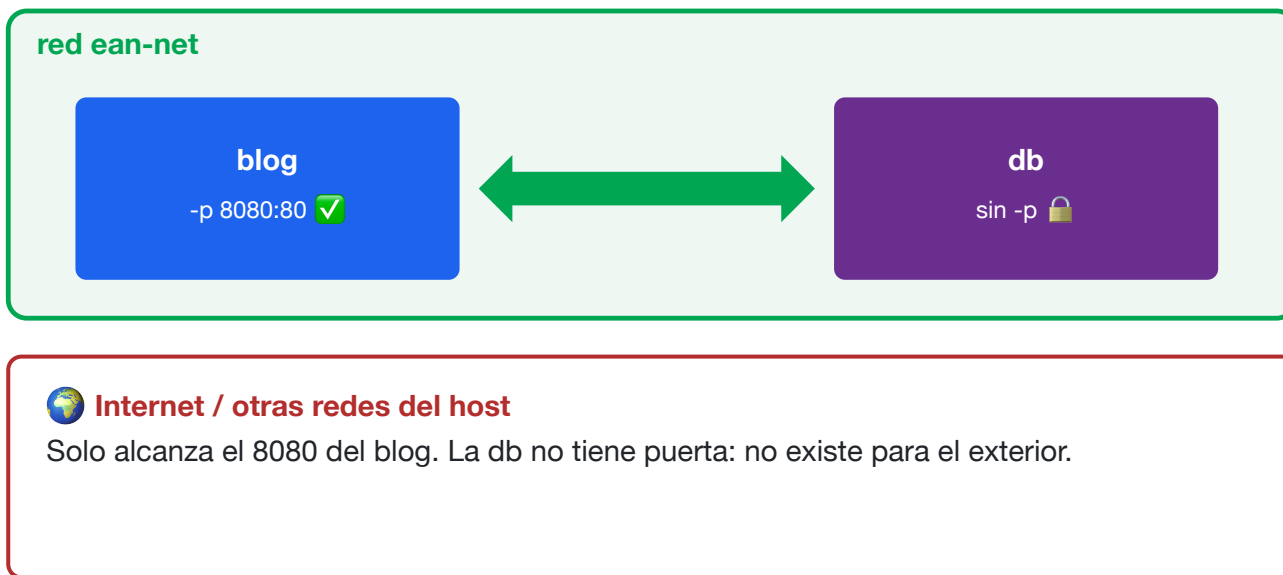
-p 8080:80 → «el puerto 8080 del host lleva al 80 del contenedor». Sin -p, NO hay puerta: el servicio existe solo dentro de la red.

EXPOSE (S2) solo documenta; -p publica de verdad. La db de hoy NO lleva -p — y esa ausencia es una decisión de seguridad.

Eje de seguridad · S3: aislar es proteger

La segmentación de red — en contenedores y en la entidad

La arquitectura segura de hoy



Los 3 principios

- 1 · Mínima exposición: publica SOLO la puerta de entrada.
- 2 · Segmentación: cada app en SU red; front y back separados si crece.
- 3 · Nada de datos al aire: la BD jamás con puerto público — ni en el lab, ni en Colpensiones.

La app de hoy: un blog con base de datos

WordPress + MySQL — dos imágenes oficiales, una red tuya

```
docker network create ean-net

docker run -d --name db --network ean-net \
  -e MYSQL_ROOT_PASSWORD=... -e MYSQL_DATABASE=wordpress ... mysql:8.0

docker run -d --name blog --network ean-net -p 8080:80 \
  -e WORDPRESS_DB_HOST=db ... wordpress:6
```

El blog encuentra la BD como «db»

WORDPRESS_DB_HOST=db — el DNS de tu red hace el resto. Ni una IP en la configuración.

Variables de entorno = configuración al RUN

Credenciales y nombres se inyectan al arrancar, no se hornean en la imagen (S2 ). En S5: secretos bien gestionados.

La BD arranca sin puerta

mysql escucha :3306 adentro, pero sin -p nadie de afuera la toca. El blog sí, por la red compartida.

Ver la red por dentro

Las herramientas de diagnóstico que usarás hoy

docker network ls

Lista las redes del host — verás bridge (fábrica) y la tuya.

docker network inspect ean-net

El plano de tu red: qué contenedores tiene, sus IPs, su subred.

docker exec blog ping db

Desde el blog, ¿alcanzo a la db por su nombre? La prueba del DNS.

docker network connect / disconnect

Conecta o desconecta un contenedor de una red EN CALIENTE.

docker logs blog

Cuando algo no conecta, los logs cuentan la historia (S1).

Regla de diagnóstico: primero la red (¿misma red?), luego el nombre (¿ping funciona?), luego el servicio (¿logs sanos?).

Lo mismo, a escala de nube

Lo que hoy haces con docker network mañana se llama VNet

Red de contenedores (hoy)

docker network create ean-net

Red virtual en Azure

VNet con subredes front / back

Red virtual en AWS

VPC con subredes públicas / privadas

DNS interno por nombre

Azure Private DNS · AWS Route 53 privado

-p: publicar solo la puerta

Load balancer / IP pública SOLO al frontend

BD sin puerto publicado

Subred privada + Network Security Group / Security Group

Segmentar apps en redes distintas

VNets/VPCs separadas, peering controlado

Los conceptos que practicas hoy son los MISMOS que gobiernan la red de la entidad en Azure — solo cambia la escala.

Los comandos de hoy

Se suman a los de las sesiones 1 y 2

docker network create <red>

Crea tu red con DNS interno incluido.

docker run --network <red> --name <n> ...

Arranca el contenedor DENTRO de tu red, con nombre estable.

docker network ls / inspect <red>

Lista redes / muestra el plano completo de una red.

docker exec <c> ping <nombre>

Prueba de alcance y DNS entre contenedores.

docker network connect <red> <c>

Suma un contenedor existente a una red (en caliente).

-e CLAVE=valor

Injecta configuración al arrancar — credenciales fuera de la imagen. 

El laboratorio de hoy

60 minutos · Play with Docker · tu app multi-contenedor

Parte 1 · Redes y DNS (15 min)

Explora la red de fábrica, crea ean-net y comprueba el superpoder: dos contenedores que se encuentran por nombre con ping — y fallan al intentarlo en la red bridge.

Parte 2 · El blog completo (30 min)

Levanta MySQL (sin puerto) y WordPress (con -p) en tu red. Instala el blog, escribe tu primer post y diagnostica la conexión con inspect, ping y logs.

Parte 3 · Comprueba el aislamiento (15 min)

Demuestra que la db es inalcanzable desde afuera y que un contenedor «intruso» en OTRA red no la ve. Cierra con el diagrama de tu arquitectura en la bitácora.

Entregable de la S3: el blog vivo (captura con tu post) + evidencia del aislamiento + diagrama de la red.

Las 3 ideas que te llevas hoy

1 · En tu propia red, los contenedores se encuentran por **NOMBRE** — nunca más una IP en la config.

2 · Sin -p no hay puerta: publica solo la entrada; el resto es tráfico interno.

3 · Aislar es proteger: la base de datos vive en la red, invisible para el mundo.

⚠ Tarea mental para la S4: tu blog de hoy guarda los posts **DENTRO** del contenedor de MySQL... ¿qué pasa si ese contenedor muere?